

คู่มือเรียน Arduino UNO Q ฉบับภาษาไทย

เอกสารเรียนแบบหนังสือ: พื้นฐานบอร์ด การต่อวงจร คำสั่ง โค้ด และแนวทาง Debug

เหมาะสำหรับ: นักศึกษา, Developer, AI Engineer, Embedded System Beginner, Maker และผู้ที่ต้องการต่อยอด IoT/AI บน Arduino UNO Q

หมายเหตุสำคัญ เอกสารฉบับแก้ไขนี้ใช้ตัวอักษรจริงในไฟล์ DOCX ทั้งหมด ไม่ใช้ภาพที่มีข้อความภาษาไทยฝังอยู่
จึงไม่เกิดปัญหาตัวอักษรเป็นช่องสี่เหลี่ยมตอนเปิดไฟล์

แหล่งข้อมูลหลัก: Newbiely Arduino UNO Q Tutorial index, Arduino official UNO Q documentation/product page

และคำอธิบายเสริมจากพื้นฐาน Arduino/Embedded Systems

สารบัญแบบอ่านเร็ว

- 1. บทที่ 1 ภาพรวม Arduino UNO Q
- 2. บทที่ 2 แผนการเรียนรู้จาก Tutorial ทั้งหมด
- 3. บทที่ 3 การติดตั้งและวิธีคิดก่อนเริ่มทำวงจร
- 4. บทที่ 4 Digital Output และ LED
- 5. บทที่ 5 Digital Input และ Button
- 6. บทที่ 6 Analog Input, PWM และ Sensor พื้นฐาน
- 7. บทที่ 7 Motor, Servo และ Actuator
- 8. บทที่ 8 Display และการแสดงผล
- 9. บทที่ 9 Communication, IoT และ AI Integration
- 10. บทที่ 10 แบบฝึกปฏิบัติละเอียด 10 บท
- 11. ภาคผนวก A Command / Code Reference
- 12. ภาคผนวก B Glossary ศัพท์เทคนิค
- 13. ภาคผนวก C Checklist Debug

วิธีใช้เอกสาร อ่านบทที่ 1-3 ก่อน จากนั้นเลือกบทปฏิบัติที่ตรงกับอุปกรณ์ที่มี แล้วใช้ภาคผนวกเป็น cheat sheet ระหว่างเขียนโค้ด

บทที่ 1: ภาพรวม Arduino UNO Q

Arduino UNO Q คือบอร์ดตระกูล Arduino ที่ออกแบบให้เป็นแพลตฟอร์มแบบ Hybrid คือมีทั้งฝั่งคอมพิวเตอร์ Linux และฝั่งไมโครคอนโทรลเลอร์สำหรับควบคุมขา I/O แบบ realtime ในบอร์ดเดียว

ส่วนของบอร์ด	หน้าที่หลัก	เหมาะกับงาน
ฝั่ง Linux / MPU	รันระบบปฏิบัติการ Debian, งานระดับสูง, Network, AI, Web, Python	Web server, IoT dashboard, computer vision, AI inference
ฝั่ง MCU	ควบคุมขา Digital/Analog, อ่าน sensor, สั่ง motor/relay แบบ realtime	อ่านปุ่ม, PWM, sensor, actuator, motor control
Bridge / RPC / Serial Concept	เชื่อมต่อข้อมูลระหว่าง Linux กับ MCU	ให้ AI หรือ Web สั่งงาน hardware จริง

คำอธิบายเสริม ให้คิดว่า UNO Q มี “สมองสองส่วน”: Linux เก่งงานหนักและเชื่อมต่อโลกภายนอก ส่วน MCU เก่งงานควบคุมขาไฟฟ้าที่ต้องตอบสนองไว

UNO Q ใช้ทำอะไรได้บ้าง

- เรียน Arduino พื้นฐาน เช่น LED, Button, Sensor, Motor
- ทำ IoT เช่น Web server, Telegram notification, dashboard

- ทำ Edge AI เช่น วิเคราะห์ภาพ/เสียงบน Linux แล้วสั่ง MCU คุมอุปกรณ์
- ทำ Robotics เช่น แขนกล, รถหุ่นยนต์, ระบบกลิ้ง+มอเตอร์
- ทำงานอุตสาหกรรมเบื้องต้น เช่น RS485/Modbus, Relay, Sensor monitoring

ข้อควรระวังเรื่องแรงดัน

- ตรวจสอบเอกสาร pinout ของบอร์ดจริงก่อนต่อวงจรทุกครั้ง
- Analog input บางบอร์ดรับสูงสุด 3.3V ไม่ควรป้อน 5V ตรง
- อุปกรณ์ที่กินกระแสสูง เช่น servo, motor, pump, lock ควรใช้แหล่งจ่ายแยก
- ต้องต่อ GND ร่วมระหว่างบอร์ดกับวงจรภายนอกเสมอเมื่อมีการส่งสัญญาณร่วมกัน

บทที่ 2: แผนการเรียนรู้จาก Tutorial ทั้งหมด

ตารางนี้เรียบเรียงหัวข้อหลักจากหน้า Tutorial index ให้เป็นมุมมองสำหรับเรียนและวางแผนโปรเจกต์ ไม่ได้คัดลอกคำอธิบายจากต้นฉบับยาว ๆ แต่จัดหมวดหมู่ให้เหมาะกับการอ่านเป็นเอกสารเรียน

หมวดหมู่	หัวข้อ	อุปกรณ์ที่ใช้	ใช้ทำอะไร	Input / Output	คำสั่งหลัก	ความยาก	ต่อยอด
พื้นฐาน	Getting Started	Arduino UNO Q, USB-C, App Lab	เริ่มเชื่อมต่อบอร์ดและทดลองโปรแกรมแรก	USB/Board -> App Lab	App Lab, Serial Monitor	เริ่มต้น	Lab ทดลองพื้นฐาน
พื้นฐาน	Communication Between Linux and MCU	UNO Q	สื่อสารระหว่างฝั่ง Linux และ MCU	Linux <-> MCU	Bridge/RPC, Serial	กลาง	Edge AI + I/O realtime
Digital Output	Blink LED	LED, resistor	เปิด/ปิดไฟ LED เป็นจังหวะ	Output	pinMode, digitalWrite, delay	ง่าย	ไฟสถานะ
Timing	Blink LED Without Delay	LED, resistor	ทำงานหลายอย่างพร้อมกันโดยไม่บล็อก	Output + Time	millis	กลาง	ระบบหลาย task
Digital Output	Blink Multiple LED	LED หลายดวง	ควบคุมไฟหลายช่อง	Output	array, for, digitalWrite	ง่าย	ไฟสถานะหลายจุด
PWM	Fade LED	LED, resistor	หรี่ไฟ LED	PWM Output	analogWrite	ง่าย	ไฟหรี่
PWM	RGB LED	RGB LED, resistor	ผสมสีไฟ	PWM Output	analogWrite	กลาง	ไฟแจ้งเตือนสี
Project	Traffic Light	LED แดง เหลือง เขียว	จำลองไฟจราจร	Output + Time	digitalWrite, delay/millis	ง่าย	ระบบสัญญาณไฟ
Digital Input	Button	Push button	อ่านปุ่มกด	Input	pinMode INPUT_PULLUP, digitalRead	ง่าย	ปุ่มควบคุม
Digital Input	Button Debounce	Push button	ลดอาการค้างของสัญญาณปุ่ม	Input + Time	millis, if	กลาง	ปุ่มที่อ่านค่าแม่นยำ
Digital Input	Switch / Limit / DIP Switch	Switch modules	อ่านสวิตช์หลายแบบ	Input	digitalRead	ง่าย	เลือกโหมดเครื่อง
Analog Input	Potentiometer	Potentiometer	อ่านค่าหมุนปรับ	Analog Input	analogRead, map	ง่าย	ปรับความเร็ว/แสง

หมวดหมู่	หัวข้อ	อุปกรณ์ที่ใช้	ใช้ทำอะไร	Input / Output	คำสั่งหลัก	ความยาก	ต่อยอด
Sensor	Rotary Encoder	Rotary encoder	อ่านทิศทางและจำนวนคลิกหมุน	Digital Input	interrupt/polling, digitalRead	กลาง	ปุ่มหมุนเมนู
Output	Piezo Buzzer / Buzzer	Buzzer	ส่งเสียงเตือน	Output	tone, noTone	ง่าย	Alarm
Motor	DC Motor	Motor driver/shield	ควบคุมมอเตอร์ DC	PWM + Direction	analogWrite, digitalWrite	กลาง	รถ/พัดลม
Motor	Servo Motor	Servo	หมุนตามองศา	PWM-like signal	Servo.h, attach, write	ง่าย	แขนกล
Sensor	Light Sensor / LDR	LDR, resistor/module	วัดแสง	Analog/Digital Input	analogRead/digitalRead	ง่าย	ไฟเปิดอัตโนมัติ
Sensor	Ultrasonic Sensor	HC-SR04	วัดระยะทาง	Trigger/Echo	pulseIn, digitalWrite	กลาง	วัดระยะ/หลบสิ่งกีดขวาง
Sensor	Motion Sensor	PIR sensor	ตรวจจับการเคลื่อนไหว	Digital Input	digitalRead	ง่าย	ไฟทางเดิน
Power Control	Relay	Relay module	สั่งเปิดปิดโหลดภายนอก	Digital Output	digitalWrite	กลาง	เปิดปิดปั๊ม/พัดลม
Actuator	Actuator with Feedback	Linear actuator	ควบคุมการเคลื่อนที่เชิงเส้น	Output + Feedback	analogRead, motor control	ยาก	ประตูก่อ/กลไก
Input	Joystick	Joystick module	อ่านแกน X/Y และปุ่ม	Analog + Digital	analogRead, digitalRead	ง่าย	รีโมทควบคุม
Sensor	Soil Moisture Sensor	Soil moisture	วัดความชื้นดิน	Analog/Digital	analogRead, map	ง่าย	รดน้ำต้นไม้
Display	LCD I2C / LCD 20x4	LCD I2C	แสดงข้อความ	I2C	Wire, LiquidCrystal_I2C	กลาง	หน้าจอระบบ

หมวดหมู่	หัวข้อ	อุปกรณ์ที่ใช้	ใช้ทำอะไร	Input / Output	คำสั่งหลัก	ความยาก	ต่อยอด
Display	OLED 128x64 / 128x32	OLED SSD1306	แสดงข้อมูลกราฟิก	I2C/SPI	Adafruit_SSD1306	กลาง	Dashboard ขนาดเล็ก
Display	7-Segment / TM1637 / 74HC595	7-seg module	แสดงตัวเลข	Digital/Serial-like	TM1637Display, shiftOut	กลาง	นาฬิกา/นับจำนวน
Display	LED Matrix	LED matrix	แสดงสัญลักษณ์/ข้อความ	Built-in/I2C/SPI	matrix API/library	กลาง	ป้ายแสดงผล

Input	Keypad 1x4 / 3x4 / 4x4	Keypad	รับรหัส/ตัวเลข	Digital Matrix	Keypad library	กลาง	ประตูรหัส
Sensor	Temperature / LM35 / DHT11 / DHT22	Temp/DHT sensor	วัดอุณหภูมิและความชื้น	Analog/Digital	analogRead, DHT library	ง่าย	สถานีอากาศ
Sensor	Touch Sensor	Touch module	ตรวจสอบสัมผัส	Digital Input	digitalRead	ง่าย	ปุ่มสัมผัส
Sensor	Door Sensor	Magnetic switch	ตรวจสอบเปิดปิดประตู	Digital Input	digitalRead	ง่าย	ระบบกันขโมย
Lock	Solenoid / Electromagnetic Lock	Lock + relay/MOSFET	ควบคุมกลอนไฟฟ้า	Output	digitalWrite	กลาง	ประตูอัจฉริยะ
RFID/NFC	RFID	RFID module	อ่านบัตร	SPI/I2C/UART	MFRC522 library	กลาง	ระบบเข้าออก
Water	Water Sensor / Rain / Valve / Pump	Water/rain sensor, pump	ตรวจน้ำและควบคุมวาล์ว/ปั๊ม	Input + Output	analogRead, relay control	กลาง	รดน้ำอัตโนมัติ
Sensor	Color Sensor	TCS3200/TCS230	ตรวจสอบสี	Digital Frequency	pulseIn	กลาง	คัดแยกสี
Sensor	Sound Sensor	Sound module	ตรวจสอบเสียง	Analog/Digital	analogRead/digitalRead	ง่าย	ตรวจสอบเสียงดัง

หมวดหมู่	หัวข้อ	อุปกรณ์ที่ใช้	ใช้ทำอะไร	Input / Output	คำสั่งหลัก	ความยาก	ตัวอย่าง
Sensor	Force Sensor	FSR	วัดแรงกด	Analog Input	analogRead, map	ง่าย	ตรวจน้ำหนักกด
Sensor	Flame Sensor	Flame sensor	ตรวจสอบเปลวไฟ	Analog/Digital	analogRead/digitalRead	ง่าย	แจ้งเตือนไฟ
Environment	Gas / MQ Sensor	MQ gas sensor	ตรวจแก๊สโดยประมาณ	Analog Input	analogRead, calibration	กลาง	เตือนแก๊ส
Sensor	Obstacle Avoidance	IR obstacle	ตรวจสอบสิ่งกีดขวาง	Digital Input	digitalRead	ง่าย	หุ่นยนต์หลบสิ่งกีดขวาง
GPS	GPS	GPS module	อ่านพิกัด	UART Serial	Serial, TinyGPS++	กลาง	ติดตามตำแหน่ง
LED	LED Strip	WS2812/NeoPixel	ควบคุมไฟเส้นหลายดวง	Digital One-wire	Adafruit_NeoPixel/FastLED	กลาง	ไฟตกแต่ง/สถานะ
Communication	RS232 / RS485 / RS422	Converter module	สื่อสารอุตสาหกรรม	UART	Serial, Modbus	ยาก	เชื่อม PLC/เครื่องจักร
IoT	Web Server	UNO Q + network	ทำหน้าที่เว็บควบคุม/แสดงผล	HTTP	Python/Arduino App Lab	กลาง	Dashboard IoT
IoT	Telegram Bot	UNO Q + internet	แจ้งเตือนผ่านแชต	Network API	HTTP client, bot token	กลาง	แจ้งเตือนอัตโนมัติ

AI	OpenClaw / AI	Camera/AI model/robot	ใช้ Linux ทำ AI แล้วสั่ง MCU คุมอุปกรณ์	Vision/Sound -> Control	Python, model runtime, Bridge/RPC	ยาก	AI robotics
----	---------------	--------------------------	--	----------------------------	--------------------------------------	-----	-------------

บทที่ 3: การติดตั้งและวิธีคิดก่อนเริ่มทำวงจร

Checklist ก่อนเสียบสาย

- รู้ว่าอุปกรณ์ใช้ไฟ 3.3V หรือ 5V
- รู้ว่า pin ที่ใช้เป็น Digital, Analog, PWM, I2C, SPI หรือ UART
- ต่อ GND ร่วมให้ถูกต้อง
- อย่าต่อโหนดกระแสสูงเข้าขา Arduino โดยตรง
- เริ่มจากตัวอย่างเล็กที่สุดก่อน แล้วค่อยรวมหลาย module

โครงสร้างโปรแกรม Arduino

```
// โครงสร้างพื้นฐานของ Arduino Sketch

void setup() {
  // ทำงานครั้งเดียวตอนบอร์ดเริ่มต้น
  // ใช้ตั้งค่า pin, Serial, library, sensor
}

void loop() {
  // ทำงานวนซ้ำตลอดเวลา
  // ใช้อ่าน input, ประมวลผล, ส่ง output
}
```

ส่วน	หน้าที่	ตัวอย่าง
ประกาศตัวแปร	กำหนด pin และค่าคงที่	const int LED_PIN = 9;
setup()	ตั้งค่าตอนเริ่ม	pinMode(), Serial.begin(), lcd.init()
loop()	งานวนซ้ำ	digitalRead(), analogRead(), digitalWrite()
function แยก	ทำให้โค้ดอ่านง่าย	readSensor(), updateDisplay()

แนวคิด Input -> Processing -> Output

ขั้น	คำถามที่ต้องตอบ	ตัวอย่าง
Input	อ่านอะไรจากโลกจริง?	ปุ่ม, แสง, ระยะ, ความชื้น
Processing	ตัดสินใจอย่างไร?	ถ้าระยะ < 20 cm ให้เตือน
Output	ส่งอะไรออกไป?	เปิด LED, ส่งเสียง, หมุน servo, แจ้งเตือนเว็บ

บทที่ 4: Digital Output และ LED

Digital Output คืออะไร

การสั่งให้ pin เป็น HIGH/LOW เพื่อควบคุม LED, relay input หรือสัญญาณ logic

คำสั่งหลัก

`pinMode(pin, OUTPUT), digitalWrite(pin, HIGH/LOW), delay(ms), millis()`

แนวทางเรียน

เริ่ม Blink LED -> Multiple LED -> RGB LED -> Traffic Light -> LED Strip

สรุปท้ายบท บทนี้ให้รู้ภาพรวมก่อนลงมือทำจริง รายละเอียดการต่อวงจรและโค้ดอยู่ในบทปฏิบัติถัดไป

บทที่ 5: Digital Input และ Button

Digital Input คืออะไร

การอ่าน logic 0/1 จากปุ่ม สวิตช์ limit switch door sensor หรือ PIR

คำสั่งหลัก

`pinMode(pin, INPUT_PULLUP), digitalRead(pin), if/else, debounce ด้วย millis()`

แนวทางเรียน

Button -> Debounce -> Multiple Button -> Switch -> Limit Switch -> DIP Switch

สรุปท้ายบท บทนี้ให้รู้ภาพรวมก่อนลงมือทำจริง รายละเอียดการต่อวงจรและโค้ดอยู่ในบทปฏิบัติถัดไป

บทที่ 6: Analog Input, PWM และ Sensor พื้นฐาน

Analog Input คืออะไร

การอ่านค่าแรงดันเป็นตัวเลข เช่น potentiometer, LDR, soil moisture

PWM คืออะไร

การเปิดปิดเร็วมากเพื่อจำลองระดับ เช่น หนี LED หรือปรับ motor speed

คำสั่งหลัก

`analogRead(), analogWrite(), map(), constrain(), Serial.println()`

สรุปท้ายบท บทนี้ให้รู้ภาพรวมก่อนลงมือทำจริง รายละเอียดการต่อวงจรและโค้ดอยู่ในบทปฏิบัติถัดไป

บทที่ 7: Motor, Servo และ Actuator

Motor/Actuator คืออะไร

อุปกรณ์ที่เปลี่ยนสัญญาณไฟฟ้าเป็นการเคลื่อนที่

ข้อควรระวัง

ห้ามจ่าย motor/lock/pump จาก pin โดยตรง ใช้ driver, relay, MOSFET และแหล่งจ่ายที่เหมาะสม

คำสั่งหลัก

Servo.h, attach(), write(), analogWrite(), digitalWrite()

สรุปท้ายบท บทนี้ให้รู้ภาพรวมก่อนลงมือทำจริง รายละเอียดการต่อวงจรและโค้ดอยู่ในบทปฏิบัติถัดไป

บทที่ 8: Display และการแสดงผล

Display ใช้ทำอะไร

แสดงสถานะ ค่าจาก sensor เมนู และผลลัพธ์ให้ผู้ใช้งานเห็น

ชนิดที่พบบ่อย

LCD I2C, OLED SSD1306, 7-Segment, LED Matrix

คำสั่งหลัก

Wire, lcd.init(), lcd.print(), display.print(), display.display()

สรุปท้ายบท บทนี้ให้รู้ภาพรวมก่อนลงมือทำจริง รายละเอียดการต่อวงจรและโค้ดอยู่ในบทปฏิบัติถัดไป

บทที่ 9: Communication, IoT และ AI Integration

Communication คืออะไร

การรับส่งข้อมูลระหว่างอุปกรณ์หรือระหว่าง Linux-MCU

IoT ใช้ทำอะไร

แสดงข้อมูลผ่านเว็บ รับคำสั่งผ่าน network แจ้งเตือนผ่าน Telegram

AI Integration

ให้ฝั่ง Linux ทำงาน AI/ภาพ/เสียง แล้วส่งผลให้ MCU ควบคุม output

สรุปท้ายบท บทนี้ให้รู้ภาพรวมก่อนลงมือทำจริง รายละเอียดการต่อวงจรและโค้ดอยู่ในบทปฏิบัติถัดไป

บทที่ 10: แบบฝึกปฏิบัติละเอียด

10.1 Arduino UNO Q - Blink LED

1. หัวข้อนี้คืออะไร

เปิดและปิด LED ตามจังหวะ เพื่อเข้าใจพื้นฐานการส่งงานขา output

2. ใช้ทำอะไรได้บ้าง

- ไฟแสดงสถานะการทำงานของ
- ไฟเตือน error
- ทดลองว่าบอร์ดและสายไฟทำงานถูกต้อง

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- LED 1 ดวง
- Resistor 220 Ω หรือ 330 Ω
- Breadboard
- Jumper wire

4. หลักการทำงาน

โปรแกรมเริ่มต้น -> กำหนด pin เป็น OUTPUT -> สั่ง HIGH ให้ LED ติด -> หน่วงเวลา -> สั่ง LOW ให้ LED ดับ -> วนซ้ำ

5. การต่อวงจร

- ขา D9 ต่อผ่าน resistor ไปยังขา Anode ของ LED
- ขา Cathode ของ LED ต่อ GND
- GND ของวงจรต้องร่วมกับบอร์ด

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
pinMode(pin, OUTPUT)	กำหนดขาให้เป็น output	ใช้ใน setup()	pinMode(9, OUTPUT);
digitalWrite(pin, HIGH/LOW)	สั่งจ่ายหรือหยุดจ่ายสัญญาณดิจิทัล	ใช้ใน loop()	digitalWrite(9, HIGH);
delay(ms)	หยุดรอเป็นมิลลิวินาที	ใช้เมื่อยอมให้โปรแกรมรอได้	delay(1000);

7. โค้ดตัวอย่าง

```
const int LED_PIN = 9; // กำหนดให้ LED ต่ออยู่ที่ขา D9

void setup() {
  // ตั้งค่าให้ขา LED_PIN เป็นขาส่งออก
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  // เปิด LED
  digitalWrite(LED_PIN, HIGH);
  delay(1000); // รอ 1 วินาที

  // ปิด LED
  digitalWrite(LED_PIN, LOW);
  delay(1000); // รอ 1 วินาที
}
```

8. อธิบายโค้ดทีละส่วน

- `const int LED_PIN = 9` คือการตั้งชื่อให้ pin เพื่อให้แก้ไขได้ง่าย
- `setup()` ทำงานครั้งเดียวตอนบอร์ดเริ่มต้น
- `loop()` ทำงานซ้ำตลอดเวลา
- `HIGH` หมายถึงส่ง logic 1 ออกไป และ `LOW` หมายถึง logic 0

9. ปัญหาที่พบบ่อย

- LED ไม่ติด: กลับขั้ว LED หรือ resistor ต่อผิด
- ติดตลอด: อาจต่อเข้ากับ 3.3V/5V ตรงโดยไม่ผ่าน pin
- กระพริบผิดจังหวะ: ตรวจสอบค่า delay

10. วิธี Debug

- ใช้ `LED_BUILTIN` ทดลองก่อน
- วัดไฟที่ pin ด้วย multimeter

- ตรวจสอบว่าขาในโค้ดตรงกับสายจริง

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Blink LED ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.2 Arduino UNO Q - Blink LED Without Delay

1. หัวข้อนี้คืออะไร

ทำไฟกระพริบโดยไม่หยุดโปรแกรม เพื่อให้ระบบอ่าน sensor หรือปุ่มไปพร้อมกันได้

2. ใช้ทำอะไรได้บ้าง

- ระบบที่ต้องอ่านปุ่มขณะไฟกระพริบ
- หุ่นยนต์ที่ต้องควบคุมมอเตอร์พร้อมกับสถานะไฟ
- งานที่มีหลาย task

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- LED
- Resistor
- Breadboard
- Jumper wire

4. หลักการทำงาน

อ่านเวลาปัจจุบันจาก millis() -> เทียบกับเวลารั้งก่อน -> ถ้าครบช่วงเวลาให้สลับสถานะ LED -> ไม่บล็อกส่วนอื่นของ loop()

5. การต่อวงจร

- ต่อ LED เหมือนบท Blink LED
- เลือกขา digital เช่น D9
- ใช้ GND ร่วมกัน

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
millis()	คืนค่าเวลาดั้งแต่บอร์ดเริ่มทำงาน	ใช้แทน delay ในงานหลาย task	unsigned long now = millis();
if (condition)	ตรวจเงื่อนไข	ใช้ตัดสินใจเมื่อครบเวลา	if (now - last >= interval)
digitalWrite()	เขียนสถานะออก pin	ใช้สลับ LED	digitalWrite(LED_PIN, ledState);

7. โค้ดตัวอย่าง

```
const int LED_PIN = 9;
const unsigned long INTERVAL = 500; // ช่วงเวลากระพริบ 0.5 วินาที

unsigned long lastToggleTime = 0; // เก็บเวลาที่สลับสถานะครั้งล่าสุด
bool ledState = false;           // false = ดับ, true = ติด

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  unsigned long now = millis();

  // ถ้าเวลาผ่านไปครบตาม INTERVAL แล้วค่อยสลับไฟ
  if (now - lastToggleTime >= INTERVAL) {
    lastToggleTime = now;
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState ? HIGH : LOW);
  }

  // สามารถเพิ่มงานอื่นตรงนี้ได้ เช่น อ่านปุ่ม อ่าน sensor หรือสื่อสาร Serial
}
```

8. อธิบายโค้ดที่ละส่วน

- lastToggleTime ใช้จำเวลาครั้งก่อน
- now - lastToggleTime ปลอดภัยกว่าการเทียบ now > lastToggleTime + interval
- loop() ไม่ถูกหยุด จึงทำงานอื่นต่อได้

9. ปัญหาที่พบบ่อย

- ไฟไม่กระพริบ: ตรวจสอบชนิดตัวแปรต้องเป็น unsigned long
- กระพริบเร็วผิดปกติ: ตรวจสอบค่า INTERVAL

- โค้ดส่วนอื่นยังซ้ำ: อาจยังมี delay() อยู่

10. วิธี Debug

- พิมพ์ค่า now และ lastToggleTime ผ่าน Serial
- ค้นหา delay() ในโปรแกรมแล้วค่อย ๆ แทนด้วย millis()

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Blink LED Without Delay ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.3 Arduino UNO Q - Button อ่านปุ่มกด

1. หัวข้อนี้คืออะไร

อ่านค่าปุ่มกดเพื่อให้ผู้ใช้สั่งงานบอร์ดได้

2. ใช้ทำอะไรได้บ้าง

- ปุ่มเปิด/ปิด LED
- ปุ่มเริ่ม/หยุดเครื่อง
- ปุ่มเลือกโหมด

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- Push button
- Jumper wire
- Breadboard

4. หลักการทำงาน

ผู้ใช้กดปุ่ม -> pin อ่านค่า LOW/HIGH -> if ตรวจสอบสถานะ -> สั่ง output ตามเงื่อนไข

5. การต่อวงจร

- ขาด้านหนึ่งของปุ่มต่อ GND
- อีกด้านต่อ D2
- ใช้ INPUT_PULLUP ในโค้ด จึงไม่ต้องใช้ resistor ภายนอก
- เมื่อไม่กดจะอ่าน HIGH และเมื่อกดจะอ่าน LOW

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
--------	----------	-----------	----------

pinMode(pin, INPUT_PULLUP)	เปิดตัวต้านทาน pull-up ภายใน	ใช้กับปุ่มต่อ GND	pinMode(2, INPUT_PULLUP);
digitalRead(pin)	อ่านค่า digital input	ใช้ใน loop()	int v = digitalRead(2);
if / else	ตัดสินใจตามสถานะปุ่ม	ใช้ควบคุม output	if (v == LOW)

7. โค้ดตัวอย่าง

```
const int BUTTON_PIN = 2;
const int LED_PIN = 9;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP); // ไม่กด = HIGH, กด = LOW
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);

  if (buttonState == LOW) {
    digitalWrite(LED_PIN, HIGH); // กดปุ่มแล้วเปิดไฟ
  } else {
    digitalWrite(LED_PIN, LOW); // ไม่กดแล้วปิดไฟ
  }
}
```

8. อธิบายโค้ดทีละส่วน

- INPUT_PULLUP ทำให้ค่า logic กลับด้านจากที่มีมือใหม่คาดไว้
- LOW ในตัวอย่างนี้หมายถึงปุ่มถูกกด
- ใช้ตัวแปร buttonState เพื่อให้อ่านโค้ดง่าย

9. ปัญหาที่พบบ่อย

- อ่านค่ากลับด้าน: เป็นผลจาก INPUT_PULLUP
- ค่าแกว่ง: สายยาวหรือไม่ต่อ GND ร่วม
- กดครั้งเดียวแต่เหมือนหลายครั้ง: ต้องใช้ debounce

10. วิธี Debug

- Serial.println(buttonState) เพื่อดูค่า 0/1
- สลับสายปุ่มแล้วทดลองใหม่
- ตรวจสอบ INPUT_PULLUP

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Button อ่านปุ่มกด ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.4 Arduino UNO Q - Potentiometer อ่านค่า Analog

1. หัวข้อนี้คืออะไร

อ่านค่าตัวต้านทานปรับค่าได้ เพื่อแปลงการหมุนเป็นตัวเลข 0-1023 โดยประมาณ

2. ใช้ทำอะไรได้บ้าง

- ปรับความสว่าง LED
- ปรับความเร็วพัดลม
- ปรับมุม Servo

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- Potentiometer 10k Ω
- Jumper wire

4. หลักการทำงาน

หมุน knob -> แรงดันที่ขากลางเปลี่ยน -> analogRead อ่านค่า -> map แปลงช่วง -> ใช้ควบคุม output

5. การต่อวงจร

- ขาด้านซ้ายต่อ 3.3V หรือ 5V ตามสเปกบอร์ด/วงจร
- ขาด้านขวาต่อ GND
- ขากลางต่อ A0
- ระวัง analog input บางบอร์ดรับแรงดันสูงสุด 3.3V

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
analogRead(pin)	อ่านค่า analog	ใช้กับ A0-Ax	int raw = analogRead(A0);
map(value, fromLow,	แปลงช่วงตัวเลข	ใช้แปลง 0-1023 เป็น 0-255	map(raw,0,1023,0,255)

fromHigh, toLow, toHigh)			
constrain(value,min,max)	จำกัดค่าตัวเลข	กั้นค่าเกินช่วง	constrain(pwm,0,255)

7. โค้ดตัวอย่าง

```
const int POT_PIN = A0;
const int LED_PIN = 9;

void setup() {
  Serial.begin(9600);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  int rawValue = analogRead(POT_PIN);      // อ่านค่า analog
  int pwmValue = map(rawValue, 0, 1023, 0, 255); // แปลงเป็นค่า PWM
  pwmValue = constrain(pwmValue, 0, 255);    // จำกัดช่วงให้ปลอดภัย

  analogWrite(LED_PIN, pwmValue);          // หนีไฟตามการหมุน

  Serial.print("raw = ");
  Serial.print(rawValue);
  Serial.print(" pwm = ");
  Serial.println(pwmValue);

  delay(50);
}
```

8. อธิบายโค้ดทีละส่วน

- analogRead คืนค่าเป็นจำนวนเต็มตาม ADC resolution
- map ช่วยแปลงช่วงค่าให้เหมาะกับ output
- Serial ใช้ตรวจว่าค่าที่อ่านเปลี่ยนจริงหรือไม่

9. ปัญหาที่พบบ่อย

- ค่าไม่เปลี่ยน: ต่อขากลางผิด
- ค่าแกว่ง: สายหลวมหรือมี noise
- LED ไม่หริ: pin ที่ใช้ต้องรองรับ PWM

10. วิธี Debug

- เปิด Serial Monitor ที่ 9600 baud
- ลองหมุนช้า ๆ แล้วดู rawValue
- ทดลองเปลี่ยนไปใช้ A1 เพื่อตรวจ pin

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Potentiometer อ่านค่า Analog ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.5 Arduino UNO Q - Servo Motor

1. หัวข้อนี้คืออะไร

ควบคุมมุมของ Servo เช่น 0-180 องศา

2. ใช้ทำอะไรได้บ้าง

- แขนกล
- กลไกเปิดปิดฝา
- หุ่นยนต์เลี้ยวล้อหน้า

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- Servo motor
- แหล่งจ่ายไฟภายนอกถ้า Servo กินกระแสสูง
- Jumper wire

4. หลักการทำงาน

กำหนดมุม -> library สร้างสัญญาณควบคุม -> Servo หมุนไปยังตำแหน่ง -> โปรแกรมวนอ่าน/ส่งค่าใหม่

5. การต่อวงจร

- สายสัญญาณ Servo ต่อ D9
- สาย VCC ต่อ 5V หรือแหล่งจ่ายที่เหมาะสม

- สาย GND ต่อ GND และต้องต่อ GND ร่วมกับ Arduino
- ถ้า Servo กระตุกให้ใช้ไฟภายนอก อย่าดึงจากบอร์ดโดยตรงมากเกินไป

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
#include <Servo.h>	นำเข้า library Servo	ใช้ก่อนประกาศ object	#include <Servo.h>
servo.attach(pin)	ผูก Servo กับ pin	ใช้ใน setup()	myServo.attach(9);
servo.write(angle)	สั่งมุม Servo	ใช้ใน loop()	myServo.write(90);

7. โค้ดตัวอย่าง

```
#include <Servo.h>

Servo myServo;
const int SERVO_PIN = 9;

void setup() {
  myServo.attach(SERVO_PIN); // กำหนด pin ควบคุม Servo
}

void loop() {
  // หมุนจาก 0 ไป 180 องศา
  for (int angle = 0; angle <= 180; angle += 5) {
    myServo.write(angle);
    delay(50);
  }

  // หมุนกลับจาก 180 ไป 0 องศา
  for (int angle = 180; angle >= 0; angle -= 5) {
    myServo.write(angle);
    delay(50);
  }
}
```

8. อธิบายโค้ดทีละส่วน

- Servo object คือสิ่งแทนตัวมอเตอร์ในโค้ด
- for loop ใช้ค่อย ๆ เพิ่มมุมให้หมุนเนียนขึ้น
- delay สั้น ๆ ช่วยให้ Servo มีเวลาขยับ

9. ปัญหาที่พบบ่อย

- Servo สั่น: ไฟไม่พอหรือ GND ไม่ร่วม
- ไม่หมุน: ต่อสายสัญญาณผิด pin
- หมุนสุดแล้วคราง: มุมเกินช่วงที่ Servo รับได้

10. วิธี Debug

- ลองสั่ง 90 องศาที่
- ใช้แหล่งจ่ายแยกและต่อ GND ร่วม
- เช็กสีสาย Servo ตามรุ่น

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Servo Motor ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.6 Arduino UNO Q - Ultrasonic Sensor วัดระยะ

1. หัวข้อนี้คืออะไร

วัดระยะด้วยการส่งคลื่นเสียงและจับเวลาคลื่นสะท้อนกลับ

2. ใช้ทำอะไรได้บ้าง

- รถหลบสิ่งกีดขวาง
- วัดระดับน้ำ
- ระบบเตือนระยะใกล้

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- Ultrasonic sensor เช่น HC-SR04
- Jumper wire
- ตัวแปลงระดับสัญญาณถ้าจำเป็น

4. หลักการทำงาน

ส่ง pulse ที่ Trigger -> Sensor ปลอยคลื่นเสียง -> Echo ยาวตามเวลาสะท้อน -> คำนวณระยะจากเวลา

5. การต่อวงจร

- VCC ต่อไฟตามสเปก sensor
- GND ต่อ GND
- TRIG ต่อ D8
- ECHO ต่อ D7
- ถ้า Echo เป็น 5V แต่บอร์ดรับ 3.3V ให้ใช้ voltage divider/level shifter

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
<code>pulseIn(pin, HIGH)</code>	วัดความยาว pulse	ใช้กับ Echo	<code>long duration = pulseIn(ECHO_PIN, HIGH);</code>
<code>digitalWrite(TRIG, HIGH/LOW)</code>	สร้าง pulse trigger	ใช้เริ่มวัดระยะ	<code>digitalWrite(TRIG_PIN, HIGH);</code>
<code>Serial.println()</code>	แสดงค่าระยะ	ใช้ debug	<code>Serial.println(distanceCm);</code>

7. โค้ดตัวอย่าง

```
const int TRIG_PIN = 8;
const int ECHO_PIN = 7;

void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

void loop() {
  // สร้างสัญญาณ trigger สั้น ๆ
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
```

```
digitalWrite(TRIG_PIN, LOW);

// อ่านระยะเวลาที่ echo เป็น HIGH
long duration = pulseIn(ECHO_PIN, HIGH, 30000); // timeout 30ms

// ความเร็วเสียงประมาณ 0.0343 cm/us และหาร 2 เพราะเดินทางไป-กลับ
float distanceCm = duration * 0.0343 / 2.0;

Serial.print("Distance: ");
Serial.print(distanceCm);
Serial.println(" cm");

delay(200);
}
```

8. อธิบายโค้ดที่ละส่วน

- Trigger ทำหน้าที่สั่งให้ sensor เริ่มวัด
- Echo เป็น pulse ที่ยาวตามระยะทาง
- หาร 2 เพราะเสียงเดินทางจาก sensor ไปวัตถุและกลับมา

9. ปัญหาที่พบบ่อย

- อ่าน 0 ตลอด: Echo ไม่กลับมาหรือ pin ผิด
- ค่าเพี้ยน: วัตถุเอียง/พื้นผิวดูดซับเสียง
- บอร์ดเสียหาย: ต่อ Echo 5V เข้าขา 3.3V โดยตรง

10. วิธี Debug

- ทดสอบกับวัตถุเรียบระยะ 10-30 cm
- พิมพ์ duration ก่อนคำนวณ
- ตรวจสอบแรงดัน Echo

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Ultrasonic Sensor วัดระยะ ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.7 Arduino UNO Q - LCD I2C แสดงข้อความ

1. หัวข้อนี้คืออะไร

แสดงข้อความและค่าจาก sensor บนจอ LCD ผ่านสาย I2C เพียง 4 เส้น

2. ใช้ทำอะไรได้บ้าง

- Dashboard อุณหภูมิ
- หน้าจอสถานะเครื่อง
- เมนูตั้งค่า

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- LCD I2C 16x2 หรือ 20x4
- Jumper wire
- Library LiquidCrystal_I2C

4. หลักการทำงาน

โปรแกรมอ่านค่า -> แปลงเป็นข้อความ -> ส่งผ่าน I2C -> LCD แสดงผล

5. การต่อวงจร

- VCC ต่อไฟตามจอที่ใช้
- GND ต่อ GND
- SDA ต่อขา SDA ของบอร์ด
- SCL ต่อขา SCL ของบอร์ด
- ที่อยู่ I2C มักเป็น 0x27 หรือ 0x3F

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
#include <Wire.h>	ใช้ระบบ I2C	ใช้กับจอ I2C	include
lcd.init()	เริ่มต้นจอ	ใช้ใน setup	lcd.init();
lcd.setCursor(col,row)	กำหนดตำแหน่งตัวอักษร	ก่อน print	lcd.setCursor(0,1);
lcd.print(text)	พิมพ์ข้อความ	แสดงผล	lcd.print("Hello");

7. โค้ดตัวอย่าง

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>
```



```
LiquidCrystal_I2C lcd(0x27, 16, 2); // address, columns, rows
```

```
void setup() {  
  lcd.init();    // เริ่มต้น LCD  
  lcd.backlight(); // เปิดไฟหลังจอ  
  
  lcd.setCursor(0, 0);  
  lcd.print("Arduino UNO Q");  
  lcd.setCursor(0, 1);  
  lcd.print("LCD I2C Ready");  
}  
  
void loop() {  
  // ตัวอย่างนี้ยังไม่ต้องทำอะไรซ้ำ  
}
```

8. อธิบายโค้ดทีละส่วน

- LiquidCrystal_I2C lcd(...) คือการสร้าง object ของจอ
- address ต้องตรงกับ module จริง
- setCursor เริ่มนับจาก 0 ทั้ง column และ row

9. ปัญหาที่พบบ่อย

- จอไม่ขึ้น: address ผิด
- เห็นแต่แสงหลังจอ: ปรับ contrast ด้านหลัง module
- ตัวอักษรเพี้ยน: library ไม่ตรงรุ่น

10. วิธี Debug

- ใช้ I2C scanner หา address
- ตรวจ SDA/SCL
- ลองเปลี่ยน 0x27 เป็น 0x3F

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ LCD I2C แสดงข้อความ ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.8 Arduino UNO Q - Relay ควบคุมโหลดภายนอก

1. หัวข้อนี้คืออะไร

ใช้ Arduino สั่งเปิดปิดอุปกรณ์ภายนอก เช่น บั๊ม พัดลม หลอดไฟ ผ่าน relay module

2. ใช้ทำอะไรได้บ้าง

- เปิดปั๊มน้ำอัตโนมัติ
- ควบคุมพัดลมตามอุณหภูมิ
- ตัดต่อโหลดแยกจากวงจร Arduino

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- Relay module
- โหลดที่ต้องการควบคุม
- แหล่งจ่ายไฟโหลด
- Jumper wire

4. หลักการทำงาน

เงื่อนไขในโปรแกรมเป็นจริง -> สั่ง pin -> relay เปลี่ยนหน้าสัมผัส -> โหลดเปิด/ปิด

5. การต่อวงจร

- VCC/GND ของ relay module ต่อกับไฟตามสเปก
- IN ต่อ D6
- ฝั่ง COM/NO/NC ต่อกับโหลดตามคู่มือ module
- งานไฟฟ้าควรให้ผู้มีความรู้ไฟฟ้าช่วยตรวจ เพราะอันตราย

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
digitalWrite(RELAY_PIN, HIGH/LOW)	สั่ง relay	ขึ้นกับ module ว่า active HIGH หรือ active LOW	digitalWrite(6, LOW);
pinMode(RELAY_PIN, OUTPUT)	กำหนดขาควบคุม	ใช้ใน setup	pinMode(6, OUTPUT);

7. โค้ดตัวอย่าง

```
const int RELAY_PIN = 6;

void setup() {
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, HIGH); // หลาย module เป็น active LOW: HIGH = ปิด
}

void loop() {
  digitalWrite(RELAY_PIN, LOW); // เปิด relay
  delay(3000);
  digitalWrite(RELAY_PIN, HIGH); // ปิด relay
  delay(3000);
}
```

8. อธิบายโค้ดทีละส่วน

- Relay module บางรุ่น active LOW จึงสั่ง LOW แล้วเปิด
- ควรตั้งสถานะเริ่มต้นใน setup เพื่อไม่ให้โหนดเปิดเอง
- ฝั่งสัญญาณกับฝั่งโหนดเป็นคนละส่วน

9. ปัญหาที่พบบ่อย

- เปิดปิดกลับด้าน: module เป็น active LOW
- รีเซ็ตเมื่อ relay ทำงาน: ไฟไม่พอหรือ noise
- โหนดไม่ทำงาน: ต่อ COM/NO/NC ผิด

10. วิธี Debug

- ฟังเสียงคลิก relay
- วัดหน้าสัมผัสด้วย multimeter
- ทดลอง LED แทนโหนดก่อน

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Relay ควบคุมโหนดภายนอก ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.9 Arduino UNO Q - DHT11 / DHT22 วัดอุณหภูมิและความชื้น

1. หัวข้อนี้คืออะไร

อ่านอุณหภูมิและความชื้นจาก sensor ตระกูล DHT เพื่อนำไปแสดงผลหรือควบคุมอุปกรณ์

2. ใช้ทำอะไรได้บ้าง

- สถานีตรวจอากาศ
- เปิดพัดลมเมื่ออุณหภูมิสูง
- บันทึกสภาพแวดล้อม

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- DHT11 หรือ DHT22
- Resistor pull-up ถ้า module ไม่มีในตัว
- Library DHT sensor library

4. หลักการทำงาน

Sensor วัดค่า -> Arduino อ่านผ่าน data pin -> library แปลงข้อมูล -> โปรแกรมตรวจค่า/แสดงผล

5. การต่อวงจร

- VCC ต่อไฟตามสเปก module
- GND ต่อ GND
- DATA ต่อ D4
- ถ้าเป็น sensor เปล่าให้ใส่ resistor pull-up ระหว่าง VCC กับ DATA

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
#include <DHT.h>	เรียกใช้ library	ใช้กับ DHT	include
dht.begin()	เริ่ม sensor	ใช้ใน setup	dht.begin();
readTemperature()	อ่านอุณหภูมิ	ใช้ใน loop	float t = dht.readTemperature();
readHumidity()	อ่านความชื้น	ใช้ใน loop	float h = dht.readHumidity();

7. โค้ดตัวอย่าง

```
#include <DHT.h>
```

```
#define DHT_PIN 4
```

```

#define DHT_TYPE DHT22 // เปลี่ยนเป็น DHT11 ถ้าใช้ DHT11

DHT dht(DHT_PIN, DHT_TYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("อ่านค่า DHT ไม่สำเร็จ");
    delay(2000);
    return;
  }

  Serial.print("Temp: ");
  Serial.print(temperature);
  Serial.print(" C, Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

  delay(2000);
}

```

8. อธิบายโค้ดทีละส่วน

- DHT_TYPE ต้องตรงกับ sensor จริง
- isnan ใช้ตรวจว่าค่าที่อ่านมาไม่ถูกต้อง
- DHT ไม่ควรอ่านถี่เกินไป จึงหน่วง 2 วินาที

9. ปัญหาที่พบบ่อย

- อ่านค่า NAN: สาย DATA ผิดหรือไม่มี pull-up
- ค่าอัปเดตช้า: เป็นข้อจำกัดของ DHT
- ค่าคลาดเคลื่อน: ตำแหน่ง sensor ใกล้ความร้อน

10. วิธี Debug

- ตรวจสอบ DHT_TYPE
- ลองตัวอย่างจาก library
- ใช้สายสั้นลง

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ DHT11 / DHT22 วัดอุณหภูมิและความชื้น ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

10.10 Arduino UNO Q - Web Server / IoT เบื้องต้น

1. หัวข้อนี้คืออะไร

ทำให้ Arduino UNO Q ส่งข้อมูลหรือรับคำสั่งผ่านเครือข่าย โดยใช้ความสามารถฝั่ง Linux/Network

2. ใช้ทำอะไรได้บ้าง

- Dashboard ดูค่า sensor
- หน้าเว็บเปิดปิดอุปกรณ์
- ระบบแจ้งเตือนผ่านอินเทอร์เน็ต

3. อุปกรณ์ที่ต้องใช้

- Arduino UNO Q
- Wi-Fi หรือ Ethernet
- Sensor/Output ตามงาน
- App Lab หรือสภาพแวดล้อม Python/Arduino ที่รองรับ

4. หลักการทำงาน

Sensor/MCU อ่านค่า realtime -> ส่งข้อมูลให้ฝั่ง Linux -> Web server แสดงผลหรือรับคำสั่ง -> ส่งคำสั่งกลับ MCU

5. การต่อวงจร

- ฝั่ง network ไม่ต้องต่อวงจรเพิ่มถ้าใช้ Wi-Fi
- sensor ต่อกับ MCU pins ตามบทที่เกี่ยวข้อง

- ระวังแรงดัน sensor และ GND ร่วม

6. คำสั่ง / Function ที่ใช้

คำสั่ง	ความหมาย	ใช้ตอนไหน	ตัวอย่าง
Serial / Bridge / RPC	ช่องทางสื่อสารระหว่างงานระดับสูงกับ MCU	ใช้เชื่อม Linux กับ IO	แนวคิดขึ้นกับ framework
HTTP GET/POST	รับส่งข้อมูลผ่านเว็บ	ใช้ทำ API/dashboard	GET /status
JSON	รูปแบบข้อมูล	ใช้ส่งค่า sensor	{"temp": 28.5}

7. โค้ดตัวอย่าง

```
// ตัวอย่างแนวคิดฝัง MCU: ส่งค่าผ่าน Serial ให้ฝั่ง Linux/Web server อ่าน
const int SENSOR_PIN = A0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(SENSOR_PIN);
  Serial.print("{\"sensor\":");
  Serial.print(raw);
  Serial.println("}");
  delay(1000);
}
```

8. อธิบายโค้ดทีละส่วน

- UNO Q เหมาะกับ IoT เพราะมีทั้ง MCU สำหรับอ่าน pin และ Linux สำหรับ network/high-level app
- JSON ช่วยให้ฝั่งเว็บอ่านข้อมูลเป็นโครงสร้างได้ง่าย
- งานจริงควรแยกหน้าที่ MCU กับ Linux ให้ชัด

9. ปัญหาที่พบบ่อย

- เว็บไม่เห็นข้อมูล: baud rate หรือช่องทางสื่อสารไม่ตรง
- ต่อ Wi-Fi ไม่ได้: ตรวจสอบ SSID/password
- ข้อมูล delay: ส่งข้อมูลถี่เกินไปหรือมี blocking code

10. วิธี Debug

- เริ่มจาก Serial ให้แน่ใจก่อน
- ทดสอบ API ด้วย browser/curl
- แยกทดสอบ sensor, network, UI ทีละส่วน

11. สรุปท้ายบท

บทนี้สอนให้เข้าใจ Web Server / IoT เบื้องต้น ตั้งแต่แนวคิด การต่อวงจร คำสั่งหลัก โค้ด และวิธีตรวจปัญหา

ภาคผนวก A: Command / Code Reference

คำสั่ง	ความหมาย	ใช้เมื่อไหร่	ตัวอย่าง
pinMode(pin, mode)	กำหนดชนิดของ pin	setup()	pinMode(9, OUTPUT);
digitalWrite(pin, value)	เขียนค่า HIGH/LOW	ควบคุม output	digitalWrite(9, HIGH);
digitalRead(pin)	อ่านค่า digital	อ่านปุ่ม/sensor digital	int v = digitalRead(2);
analogRead(pin)	อ่านค่า analog	อ่าน sensor analog	int raw = analogRead(A0);
analogWrite(pin, value)	ส่ง PWM	หรือ LED/ปรับ speed	analogWrite(9, 128);
delay(ms)	หน่วงเวลาแบบบล็อก	งานง่าย ๆ	delay(1000);
millis()	อ่านเวลาที่ผ่านไป	งานไม่บล็อก	unsigned long now = millis();
Serial.begin(baud)	เริ่ม Serial	debug	Serial.begin(9600);
Serial.print/println	พิมพ์ค่า	debug/status	Serial.println(value);
map()	แปลงช่วงค่า	analog -> PWM	map(raw,0,1023,0,255);
constrain()	จำกัดช่วงค่า	กันค่าหลุด	constrain(x,0,255);
Servo.attach()	ผูก Servo กับ pin	setup servo	servo.attach(9);
Servo.write()	สั่งมุม Servo	control servo	servo.write(90);
Wire	I2C bus	LCD/OLED/sensor I2C	#include <Wire.h>

ภาคผนวก B: Glossary ศัพท์เทคนิค

ศัพท์	ชื่อเต็ม/คำแปล	ความหมายแบบเข้าใจง่าย
MCU	Microcontroller Unit / ไมโครคอนโทรลเลอร์	ชิปควบคุมขา I/O แบบ realtime
MPU	Microprocessor Unit / ไมโครโปรเซสเซอร์	ชิปที่รันระบบปฏิบัติการ เช่น Linux
GPIO	General Purpose Input/Output	ขาดิจิทัลทั่วไปที่กำหนดเป็น input หรือ output ได้
PWM	Pulse Width Modulation / การปรับความกว้างพัลส์	เทคนิคควบคุมระดับเฉลี่ยของสัญญาณ

I2C	Inter-Integrated Circuit	บัสสื่อสาร 2 เส้น SDA/SCL
UART	Universal Asynchronous Receiver/Transmitter	Serial TX/RX
SPI	Serial Peripheral Interface	บัสความเร็วสูงสำหรับ module หลายชนิด
Debounce	การลดสัญญาณต่ง	ทำให้ปุ่มกดอ่านค่าไม่ซ้ำผิดพลาด
Pull-up	การดึงสัญญาณขึ้น	ทำให้ input มีค่าเริ่มต้นชัดเจน
GND common	กราวด์ร่วม	ทำให้วงจรมีจุดอ้างอิงแรงดันเดียวกัน

ภาคผนวก C: Checklist Debug

อาการ	สาเหตุที่พบบ่อย	วิธีตรวจ
Serial Monitor ไม่ขึ้น	baud rate ไม่ตรง, เลือก port ผิด	ตรวจ Tools > Port และค่า Serial.begin
LED ไม่ติด	กลับขั้ว, resistor/pin ผิด	ใช้ blink built-in ทดสอบก่อน
Sensor อ่านค่าเพี้ยน	ไฟเลี้ยงไม่ถูก, สายหลวม, noise	พิมพ์ raw value ด้วย Serial
ปุ่มอ่านค่ากลับด้าน	ใช้ INPUT_PULLUP	จำว่าไม่กด=HIGH กด=LOW
Servo กระตุก	ไฟไม่พอหรือ GND ไม่ร่วม	ใช้แหล่งจ่ายแยกและต่อ GND ร่วม
I2C display ไม่แสดง	address ผิด, SDA/SCL ผิด	รัน I2C scanner
Relay ทำงานกลับด้าน	module เป็น active LOW	ทดลอง HIGH/LOW และอ่านคู่มือ module
โปรแกรมค้าง	ใช้ delay หรือรอ input นาน	เปลี่ยนเป็น millis หรือใส่ timeout

แหล่งข้อมูลและขอบเขตการเรียนรู้

- Newbiely: Arduino UNO Q Tutorial index ใช้เป็นรายการหัวข้อสำหรับจัดหมวดการเรียนรู้
- Arduino official documentation/product page ใช้อ้างอิงภาพรวม UNO Q, Linux/Debian, Wi-Fi/Bluetooth และแนวคิด Hybrid Linux + MCU
- คำอธิบายเรื่องวงจร โค้ด Debug และข้อควรระวัง เป็นคำอธิบายเสริมจากพื้นฐาน Arduino/Embedded Systems เพื่อให้เอกสารใช้อ่านเรียนได้ต่อเนื่อง

ข้อควรจำ ก่อนต่ออุปกรณ์จริง ให้ตรวจ pinout, แรงดัน, กระแส และคู่มือ module รุ่นที่ใช้ทุกครั้ง เพราะ module คนละรุ่นอาจมี pin หรือ logic active HIGH/LOW ต่างกัน